In [ ]:
```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
```

```
[]
```

In [1]:
```
from google.colab import files
files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving kaggle.json to kaggle.json
```

Out[1]:
```
{'kaggle.json': b'{"username":"rozii123","key":"95a03ba05fb47a7cc7a40c49fb7dfbe
6"}'}
```

In [2]:
```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

In [3]:
```
!kaggle datasets list
```

```
ref                                                                title
size  lastUpdated                 downloadCount  voteCount  usabilityRating
------------------------------------------------------------  --------------------
----------------------------  ----------  --------------------------  -------------
---------  ---------------
dmahajanbe23/bmw-global-automotive-sales                           BMW Global Automotiv
e Sales                              55017  2026-02-22 18:18:38.170000          10082
202                1
ssssws/chocolate-sales-dataset-2023-2024                           Chocolate Sales Data
set 2023 - 2024                   24420255  2026-03-07 04:58:02.387000           3073
58                1
grandmaster07/student-exam-performance-dataset-analysis            Student Exam Perform
ance Dataset Analysis                96178  2026-02-10 09:47:13.350000           6351
114                1
shree0910/ai-and-data-science-job-market-dataset-20202026          AI & Data Science Jo
b Market Dataset (2020–2026)       188367  2026-03-09 07:40:43.393000           2029
52                1
alitaqishah/spotify-wrapped-2025-top-songs-and-artists             Spotify Wrapped 2025
| Top Songs & Artists                8564  2026-03-13 20:37:00.967000            507
30                1
jayjoshi37/sleep-screen-time-and-stress-analysis                   Sleep, Screen Time a
nd Stress Analysis                 310158  2026-03-11 14:21:59.017000           1419
29                1
thuandao/superstore-sales-analytics                                SuperStore Sales Ana
lytics                            2283450  2026-03-06 12:31:25.800000           2534
52                1
muhammadaqeelkabir/top-anime-csv                                   Top-anime.csv
220660  2026-03-08 09:27:48.840000            825         26                1
rhythmghai/resume-screening-dataset-200k-candidates                💼 Resume Screening
Dataset (200K Candidates)          3750840  2026-03-15 08:54:08.510000            26
3         22                1
rhythmghai/netflix-user-watching-behavior-dataset                  🍿 Netflix User Wat
ching Behavior Dataset              1149386  2026-03-15 10:21:18.723000             9
19         29                1
jayjoshi37/smartphone-usage-and-addiction-prediction               Smartphone Usage & A
ddiction Prediction                182295  2026-02-19 05:25:36.980000           1102
29                1
lucalullo/global-suicide-rates-by-country-2000-2021                global-suicide-rates
-by-country-2000-2021              179429  2026-03-12 19:46:55.247000            486
27                1
amineipad/e-commerce-marketing-and-sales-revenue-prediction        E-Commerce Marketing
& Sales Revenue Prediction        1051226  2026-03-09 18:30:03.560000           1036
31                1
zkskhurram/global-petrol-prices-impact-of-2026-us-iran-war         Global Petrol Prices
— Impact of 2026 US-Iran War        8233  2026-03-09 17:20:10.313000           1247
28                1
suhanigupta04/student-placement-prediction-dataset                 Student Placement Pr
ediction Dataset                  1873115  2026-03-02 07:41:44.213000           2212
43                1
andrewdevila/movie-vs-emotional-attachment-survey                  Movie vs Emotional A
ttachment Survey                   233858  2026-03-06 18:50:23.407000            500
28                1
dhrubangtalukdar/startup-funding-and-outcome-dataset               Startup Funding and
Outcome Dataset                   3968395  2026-03-09 17:26:46.180000            312
25                1
alitaqishah/nvidia-stock-data-19992026-the-ai-mega-stock           NVIDIA Stock Data (1
```

```
999–2026) | The AI Mega Stock      241455  2026-03-12 19:50:33.480000            395
35                1
muhammadaammartufail/global-e-commerce-sales-and-customer-data  Global E-Commerce Sa
les & Customer Data                 62340  2026-03-11 15:40:59.833000            236
55                1
algozee/netflix-content-analysis                                Netflix Content Anal
ysis                             1400865  2026-03-11 18:49:11.850000            545
34                1
```

In [4]:
```
!kaggle datasets download -d jessicali9530/celeba-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/jessicali9530/celeba-dataset
License(s): other
Downloading celeba-dataset.zip to /content
100% 1.33G/1.33G [00:34<00:00, 41.9MB/s]
```

In [6]:
```
!unzip -q celeba-dataset.zip
```

```
replace img_align_celeba/img_align_celeba/000001.jpg? [y]es, [n]o, [A]ll, [N]one,
[r]ename: A
```

In [7]:
```
import os
os.listdir()
```

Out[7]:
```
['.config',
 'list_attr_celeba.csv',
 'celeba-dataset.zip',
 'img_align_celeba',
 'list_bbox_celeba.csv',
 'kaggle.json',
 'list_landmarks_align_celeba.csv',
 'list_eval_partition.csv',
 'sample_data']
```

In [8]:
```
# Step 1 : Importing the libraries
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.preprocessing import image
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.applications import VGG16
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

In [9]:
```
# Step 2: This is dealing with annotations, landmarks, and partition files,so
kaggle_prefix = ""   # This is for Google Colab

data = pd.read_csv("list_attr_celeba.csv")
landmarks = pd.read_csv("list_landmarks_align_celeba.csv")
partition = pd.read_csv("list_eval_partition.csv")
```

```
print(data.shape, landmarks.shape, partition.shape)
data.head()
```

```
(202599, 41) (202599, 11) (202599, 2)
```

Out[9]:

	image_id	5_o_Clock_Shadow	Arched_Eyebrows	Attractive	Bags_Under_Eyes	Bald	Ba
0	000001.jpg	-1	1	1	-1	-1	
1	000002.jpg	-1	-1	-1	1	-1	
2	000003.jpg	-1	-1	-1	-1	-1	
3	000004.jpg	-1	-1	1	-1	-1	
4	000005.jpg	-1	1	1	-1	-1	

5 rows × 41 columns

In [13]:

```
# Step 3:Using the Helper functions

def make_binary(series):
    return series.replace({-1: 0, 1: 1})

def build_base_dataframe(target_columns):
    selected = ['image_id'] + target_columns
    df_targets = data[selected].copy()

    for col in target_columns:
        df_targets[col] = make_binary(df_targets[col])

    df = pd.merge(df_targets, partition, on='image_id', how='inner')
    return df

def add_mouth_width_quartile(df):
    lm = landmarks[['image_id', 'leftmouth_x', 'rightmouth_x']].copy()
    lm['mouth_width'] = lm['rightmouth_x'] - lm['leftmouth_x']
    df = pd.merge(df, lm[['image_id', 'mouth_width']], on='image_id', how='inner')

    # 4 quartiles: 0,1,2,3
    df['mouth_width_quartile'] = pd.qcut(
        df['mouth_width'],
        q=4,
        labels=False,
        duplicates='drop'
    )
    df['mouth_width_quartile'] = df['mouth_width_quartile'].astype(int)
    return df

def split_partitions(df):
    # Correct partition meaning in CelebA (following the starter code order)
    # 0 = train, 1 = test, 2 = validation
    train_df = df[df['partition'] == 0].reset_index(drop=True)
    test_df  = df[df['partition'] == 1].reset_index(drop=True)
    val_df   = df[df['partition'] == 2].reset_index(drop=True)
    return train_df, val_df, test_df
```

```
def sample_partitions(train_df, val_df, test_df, train_frac=0.1, val_n=1000, test_n
    train_df = train_df.sample(frac=train_frac, random_state=random_state).reset_in
    val_df   = val_df.sample(n=min(val_n, len(val_df)), random_state=random_state).
    test_df  = test_df.sample(n=min(test_n, len(test_df)), random_state=random_stat
    return train_df, val_df, test_df

def plot_confusion(cm, labels, title):
    plt.figure(figsize=(6,5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(title)
    plt.show()
```

In [14]:

```
# Step 4: Datagenerator on the basis of given starter code
class DataGenerator(tf.keras.utils.Sequence):

    def __init__(self, df, target_cols, batch_size=32, dim=(218,178), n_channels=3,
                 include_metadata=False, metadata_col=None, metadata_dim=None, shuf
        self.dim = dim
        self.batch_size = batch_size
        self.df = df.reset_index(drop=True)
        self.target_cols = target_cols
        self.n_classes = len(target_cols)
        self.n_channels = n_channels
        self.include_metadata = include_metadata
        self.metadata_col = metadata_col
        self.metadata_dim = metadata_dim
        self.shuffle = shuffle
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(len(self.df) / self.batch_size))

    def __getitem__(self, index):
        indexes = self.indexes[index*self.batch_size:(index+1)*self.batch_size]
        df_temp = self.df.iloc[indexes].reset_index(drop=True)
        return self.__data_generation(df_temp)

    def on_epoch_end(self):
        self.indexes = np.arange(len(self.df))
        if self.shuffle:
            np.random.shuffle(self.indexes)

    def __data_generation(self, df_temp):
        current_batch_size = len(df_temp)
        X = np.empty((current_batch_size, *self.dim, self.n_channels), dtype=np.flo
        y = np.empty((current_batch_size, self.n_classes), dtype=np.float32)

        if self.include_metadata:
            meta = np.empty((current_batch_size, self.metadata_dim), dtype=np.float

        for i, row in df_temp.iterrows():
            img_path = os.path.join(kaggle_prefix, "img_align_celeba", "img_align_c
```

```
            img = image.load_img(img_path, target_size=self.dim)
            img = image.img_to_array(img) / 255.0
            X[i] = img
            y[i] = row[self.target_cols].astype(float).values

            if self.include_metadata:
                quartile = int(row[self.metadata_col])
                one_hot = np.zeros(self.metadata_dim, dtype=np.float32)
                one_hot[quartile] = 1.0
                meta[i] = one_hot

        if self.include_metadata:
            return (X, meta), y
        return X, y
```

In [15]:

```
# Step 5: Model building process

def build_image_only_model(output_units):
    image_input = tf.keras.Input(shape=(218, 178, 3))
    base_model = VGG16(input_shape=(218, 178, 3), include_top=False, weights='image
    base_model.trainable = False

    x = base_model(image_input, training=False)
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(1024, activation='relu')(x)
    output = tf.keras.layers.Dense(output_units, activation='sigmoid')(x)

    model = tf.keras.Model(inputs=image_input, outputs=output)
    model.compile(
        optimizer=SGD(learning_rate=0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

def build_image_plus_metadata_model(output_units, metadata_dim=4):
    image_input = tf.keras.Input(shape=(218, 178, 3), name='image_input')
    metadata_input = tf.keras.Input(shape=(metadata_dim,), name='metadata_input')

    base_model = VGG16(input_shape=(218, 178, 3), include_top=False, weights='image
    base_model.trainable = False

    x = base_model(image_input, training=False)
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(1024, activation='relu')(x)

    merged = tf.keras.layers.Concatenate()([x, metadata_input])
    merged = tf.keras.layers.Dense(128, activation='relu')(merged)
    output = tf.keras.layers.Dense(output_units, activation='sigmoid')(merged)

    model = tf.keras.Model(inputs=[image_input, metadata_input], outputs=output)
    model.compile(
        optimizer=SGD(learning_rate=0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy']
```

```
    )
    return model
```

In [16]:
```
# Step 6 : Implementing on evaluation helpers

def evaluate_binary_model(model, generator, true_df, target_col, title):
    probs = model.predict(generator)
    preds = (probs[:, 0] > 0.5).astype(int)
    true = true_df[target_col].values.astype(int)

    acc = accuracy_score(true, preds)
    cm = confusion_matrix(true, preds)

    print(f"{title} Accuracy: {acc:.4f}")
    print(classification_report(true, preds, digits=4))
    plot_confusion(cm, labels=[0,1], title=title)
    return acc, cm

def evaluate_multitarget_4class(model, generator, true_df, target_cols, title):
    probs = model.predict(generator)
    preds_bin = (probs > 0.5).astype(int)

    true_gender = true_df[target_cols[0]].values.astype(int)
    true_age = true_df[target_cols[1]].values.astype(int)

    pred_gender = preds_bin[:, 0]
    pred_age = preds_bin[:, 1]

    true_4 = true_gender * 2 + true_age
    pred_4 = pred_gender * 2 + pred_age

    labels = ['Female-Old', 'Female-Young', 'Male-Old', 'Male-Young']
    acc = accuracy_score(true_4, pred_4)
    cm = confusion_matrix(true_4, pred_4, labels=[0,1,2,3])

    print(f"{title} Accuracy: {acc:.4f}")
    plot_confusion(cm, labels=labels, title=title)
    return acc, cm
```

R1 - Male VS Female

In [17]:
```
# Step 7: R1 data preparations
r1_df = build_base_dataframe(['Male'])

train_df, val_df, test_df = split_partitions(r1_df)
train_df, val_df, test_df = sample_partitions(train_df, val_df, test_df, train_frac

print(len(train_df), len(val_df), len(test_df))
train_df.head()
```

```
16277 1000 1000
```

Out[17]:

	image_id	Male	partition
0	056354.jpg	1	0
1	130368.jpg	1	0
2	098887.jpg	1	0
3	039403.jpg	1	0
4	080965.jpg	0	0

In [18]:

```
# Step 8: R1 datagenerators and models
r1_train_gen = DataGenerator(train_df, target_cols=['Male'], batch_size=32, shuffle
r1_val_gen   = DataGenerator(val_df, target_cols=['Male'], batch_size=32, shuffle=F
r1_test_gen  = DataGenerator(test_df, target_cols=['Male'], batch_size=32, shuffle=

r1_model = build_image_only_model(output_units=1)
r1_model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/v
gg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 218, 178, 3)	0
vgg16 (Functional)	(None, 6, 5, 512)	14,714,688
flatten (Flatten)	(None, 15360)	0
dense (Dense)	(None, 1024)	15,729,664
dense_1 (Dense)	(None, 1)	1,025

Total params: 30,445,377 (116.14 MB)

Trainable params: 15,730,689 (60.01 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [20]:

```
# Step 9: Training the R1 model
r1_history = r1_model.fit(
    r1_train_gen,
    validation_data=r1_val_gen,
    epochs=5 )
```

```
Epoch 1/5
509/509 ━━━━━━━━━━━━━━━━━━━━ 110s 215ms/step - accuracy: 0.8391 - loss: 0.4560 - val
_accuracy: 0.8590 - val_loss: 0.4164
Epoch 2/5
509/509 ━━━━━━━━━━━━━━━━━━━━ 109s 214ms/step - accuracy: 0.8633 - loss: 0.4002 - val
_accuracy: 0.8710 - val_loss: 0.3747
Epoch 3/5
509/509 ━━━━━━━━━━━━━━━━━━━━ 109s 213ms/step - accuracy: 0.8718 - loss: 0.3673 - val
_accuracy: 0.8790 - val_loss: 0.3489
Epoch 4/5
509/509 ━━━━━━━━━━━━━━━━━━━━ 109s 214ms/step - accuracy: 0.8760 - loss: 0.3446 - val
_accuracy: 0.8860 - val_loss: 0.3311
Epoch 5/5
509/509 ━━━━━━━━━━━━━━━━━━━━ 108s 213ms/step - accuracy: 0.8804 - loss: 0.3278 - val
_accuracy: 0.8880 - val_loss: 0.3176
```

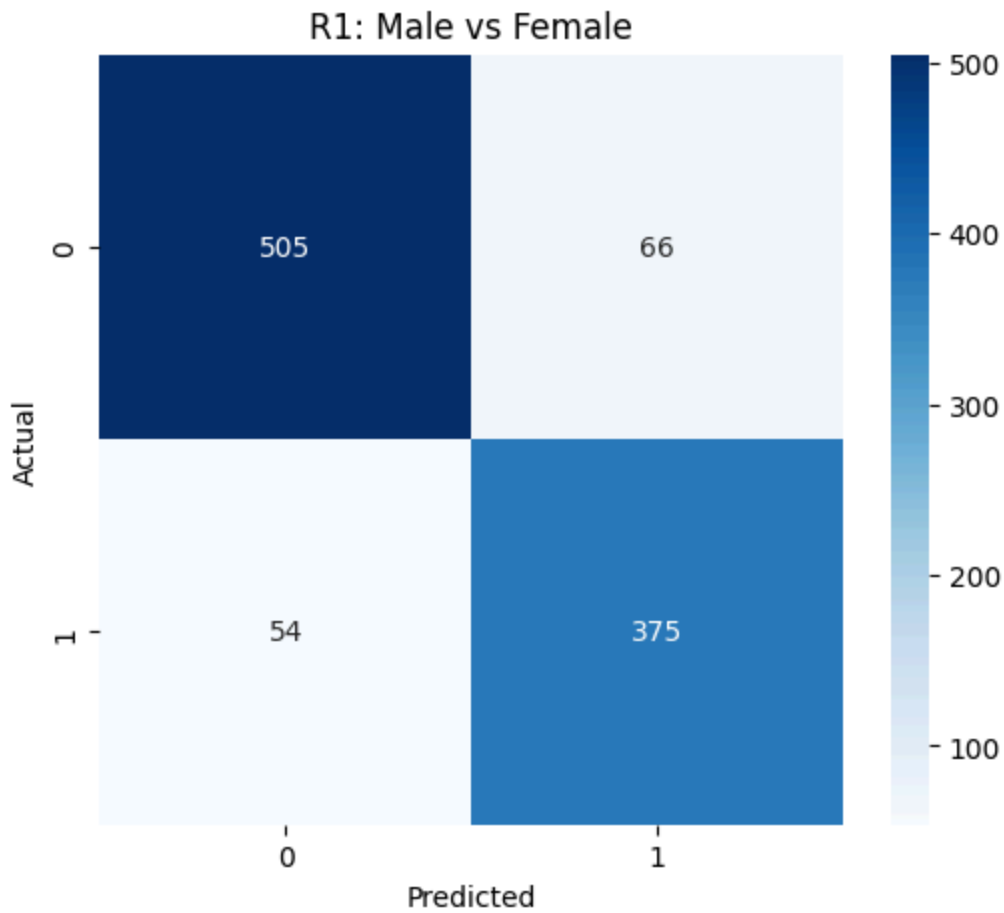
In [21]:

```python
# Step 10 : Evaluation of  R1 on the required test partition
r1_acc, r1_cm = evaluate_binary_model(
    r1_model,
    r1_test_gen,
    test_df,
    target_col='Male',
    title='R1: Male vs Female' )
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
32/32 ━━━━━━━━━━━━━━━━━━━━ 8s 228ms/step
R1: Male vs Female Accuracy: 0.8800
              precision    recall  f1-score   support

           0     0.9034    0.8844    0.8938       571
           1     0.8503    0.8741    0.8621       429

    accuracy                         0.8800      1000
   macro avg     0.8769    0.8793    0.8779      1000
weighted avg     0.8806    0.8800    0.8802      1000
```



R2- Gender and Age together

In [22]:
```
# Step 11: R2 data preparation
r2_df = build_base_dataframe(['Male', 'Young'])

train_df2, val_df2, test_df2 = split_partitions(r2_df)
train_df2, val_df2, test_df2 = sample_partitions(train_df2, val_df2, test_df2, trai

print(len(train_df2), len(val_df2), len(test_df2))
train_df2.head()
```

```
16277 1000 1000
```

Out[22]:

	image_id	Male	Young	partition
0	056354.jpg	1	1	0
1	130368.jpg	1	0	0
2	098887.jpg	1	0	0
3	039403.jpg	1	0	0
4	080965.jpg	0	1	0

In [23]:
```
# STEP 12: R2 generators and models
r2_train_gen = DataGenerator(train_df2, target_cols=['Male', 'Young'], batch_size=3
```

```
r2_val_gen   = DataGenerator(val_df2, target_cols=['Male', 'Young'], batch_size=32,
r2_test_gen  = DataGenerator(test_df2, target_cols=['Male', 'Young'], batch_size=32

r2_model = build_image_only_model(output_units=2)
r2_model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 218, 178, 3)	0
vgg16 (Functional)	(None, 6, 5, 512)	14,714,688
flatten_1 (Flatten)	(None, 15360)	0
dense_2 (Dense)	(None, 1024)	15,729,664
dense_3 (Dense)	(None, 2)	2,050

Total params: 30,446,402 (116.14 MB)

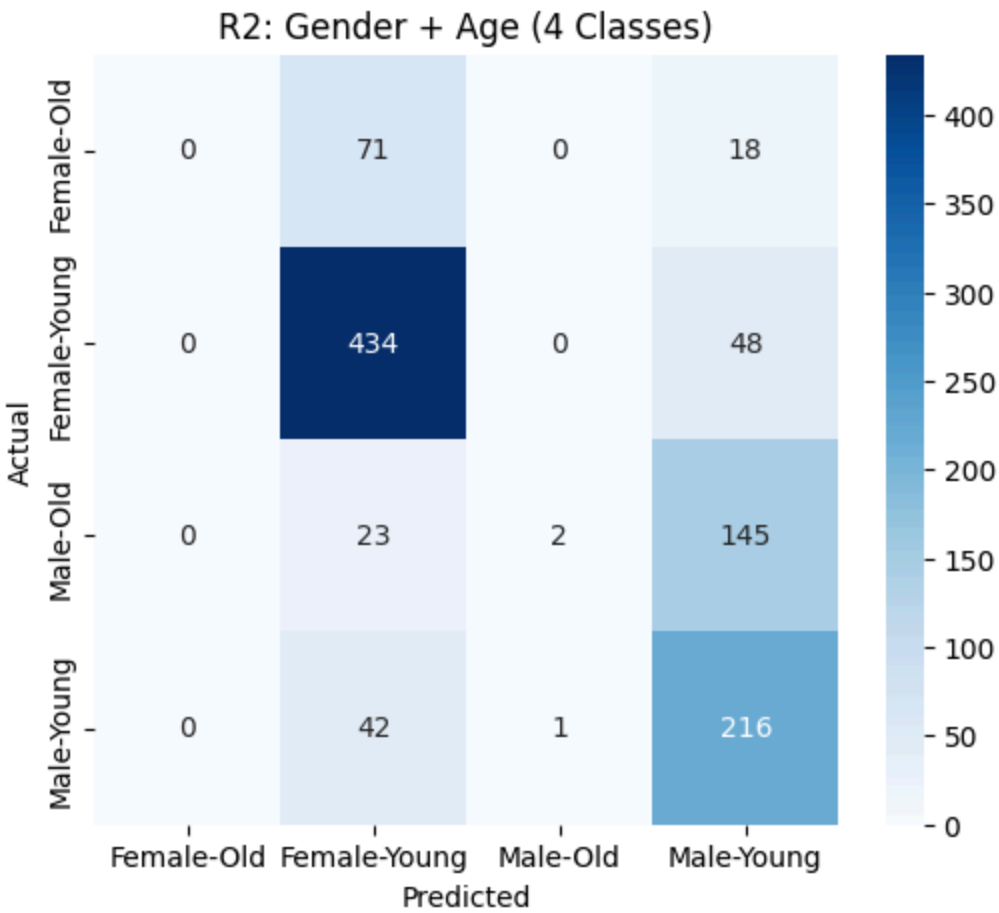
Trainable params: 15,731,714 (60.01 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [24]:
```
# Step 13: Training R2 model
r2_history = r2_model.fit(
    r2_train_gen,
    validation_data=r2_val_gen,
    epochs=5
)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
509/509 ━━━━━━━━━━━━━━━━━━━━ 0s 206ms/step - accuracy: 0.5116 - loss: 0.6047
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
```

```
▯▯▯
```



R3 (a). Baseline models smiling classification

In [26]:
```
# Step 15: R3 baseline data preparation
r3a_df = build_base_dataframe(['Smiling'])

train_df3, val_df3, test_df3 = split_partitions(r3a_df)
train_df3, val_df3, test_df3 = sample_partitions(train_df3, val_df3, test_df3, trai

print(len(train_df3), len(val_df3), len(test_df3))
train_df3.head()
```

```
16277 1000 1000
```

Out[26]:

	image_id	Smiling	partition
0	056354.jpg	0	0
1	130368.jpg	1	0
2	098887.jpg	0	0
3	039403.jpg	0	0
4	080965.jpg	0	0

In [27]:
```
# Step 16: R3(a) datagenerators and model
r3a_train_gen = DataGenerator(train_df3, target_cols=['Smiling'], batch_size=32, sh
```

```
r3a_val_gen   = DataGenerator(val_df3, target_cols=['Smiling'], batch_size=32, shuf
r3a_test_gen  = DataGenerator(test_df3, target_cols=['Smiling'], batch_size=32, shu

r3a_model = build_image_only_model(output_units=1)
r3a_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 218, 178, 3)	0
vgg16 (Functional)	(None, 6, 5, 512)	14,714,688
flatten_2 (Flatten)	(None, 15360)	0
dense_4 (Dense)	(None, 1024)	15,729,664
dense_5 (Dense)	(None, 1)	1,025

Total params: 30,445,377 (116.14 MB)

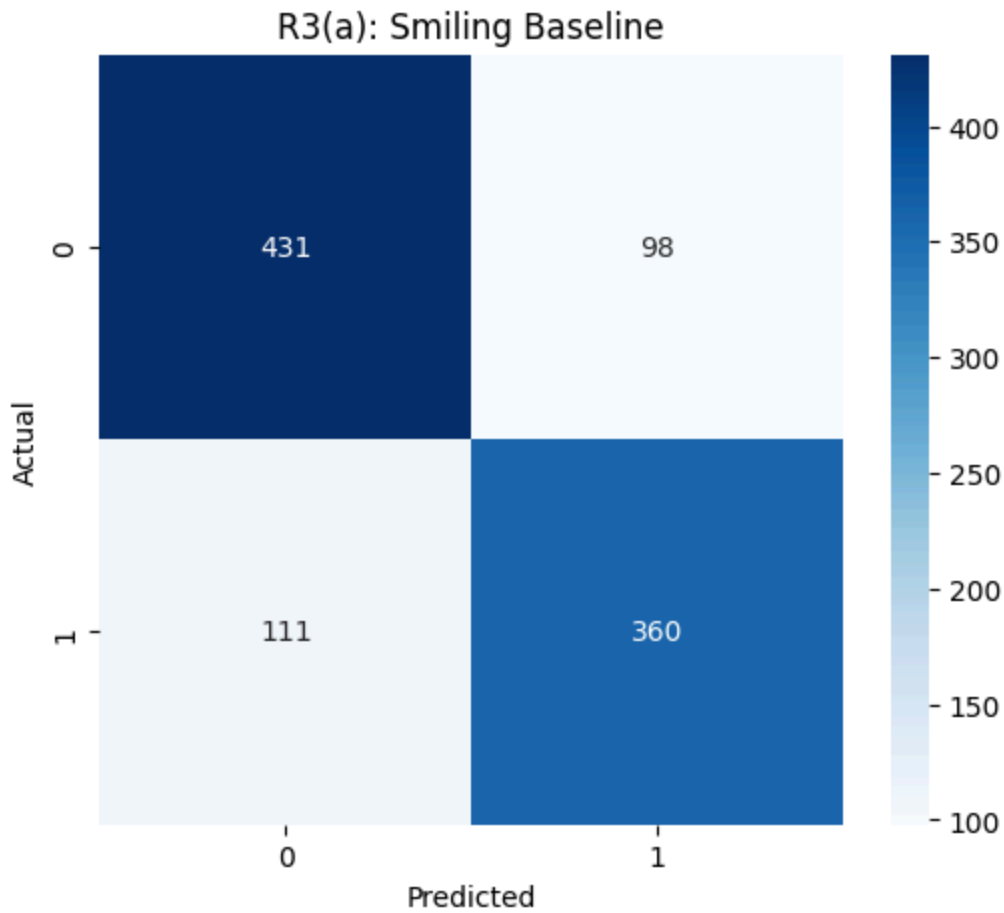
Trainable params: 15,730,689 (60.01 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [28]:
```
# STEP 17: Training R3(a) model
r3a_history = r3a_model.fit(
    r3a_train_gen,
    validation_data=r3a_val_gen,
    epochs=5
)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
509/509 ━━━━━━━━━━━━━━━━━━━━ 0s 204ms/step - accuracy: 0.5568 - loss: 0.6841
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
```

```
▯▯▯
```



R3 (b) Compute mouth width(smiling) quartile

In [32]:
```
# Step 19: R3(b) data preparation with mouth width quartile
r3b_df = build_base_dataframe(['Smiling'])
r3b_df = add_mouth_width_quartile(r3b_df)

train_df4, val_df4, test_df4 = split_partitions(r3b_df)
train_df4, val_df4, test_df4 = sample_partitions(train_df4, val_df4, test_df4, trai

print(len(train_df4), len(val_df4), len(test_df4))
train_df4[['image_id', 'Smiling', 'mouth_width', 'mouth_width_quartile']].head()
```

```
16277 1000 1000
```

Out[32]:

	image_id	Smiling	mouth_width	mouth_width_quartile
0	056354.jpg	0	5	0
1	130368.jpg	1	38	2
2	098887.jpg	0	28	0
3	039403.jpg	0	26	0
4	080965.jpg	0	37	2

In [33]:
```
# Step 20: R3(b) datagenerators and model
r3b_train_gen = DataGenerator(
    train_df4,
    target_cols=['Smiling'],
    batch_size=32,
    include_metadata=True,
    metadata_col='mouth_width_quartile',
    metadata_dim=4,
    shuffle=True
)

r3b_val_gen = DataGenerator(
    val_df4,
    target_cols=['Smiling'],
    batch_size=32,
    include_metadata=True,
    metadata_col='mouth_width_quartile',
    metadata_dim=4,
    shuffle=False
)

r3b_test_gen = DataGenerator(
    test_df4,
    target_cols=['Smiling'],
    batch_size=32,
    include_metadata=True,
    metadata_col='mouth_width_quartile',
    metadata_dim=4,
    shuffle=False
)

r3b_model = build_image_plus_metadata_model(output_units=1, metadata_dim=4)
r3b_model.summary()
```

Model: "functional_3"

Layer (type)	Output Shape	Param #	Connected to
image_input (InputLayer)	(None, 218, 178, 3)	0	-
vgg16 (Functional)	(None, 6, 5, 512)	14,714,688	image_input[0][0]
flatten_3 (Flatten)	(None, 15360)	0	vgg16[0][0]
dense_6 (Dense)	(None, 1024)	15,729,664	flatten_3[0][0]
metadata_input (InputLayer)	(None, 4)	0	-
concatenate (Concatenate)	(None, 1028)	0	dense_6[0][0], metadata_input[0…
dense_7 (Dense)	(None, 128)	131,712	concatenate[0][0]
dense_8 (Dense)	(None, 1)	129	dense_7[0][0]

Total params: 30,576,193 (116.64 MB)

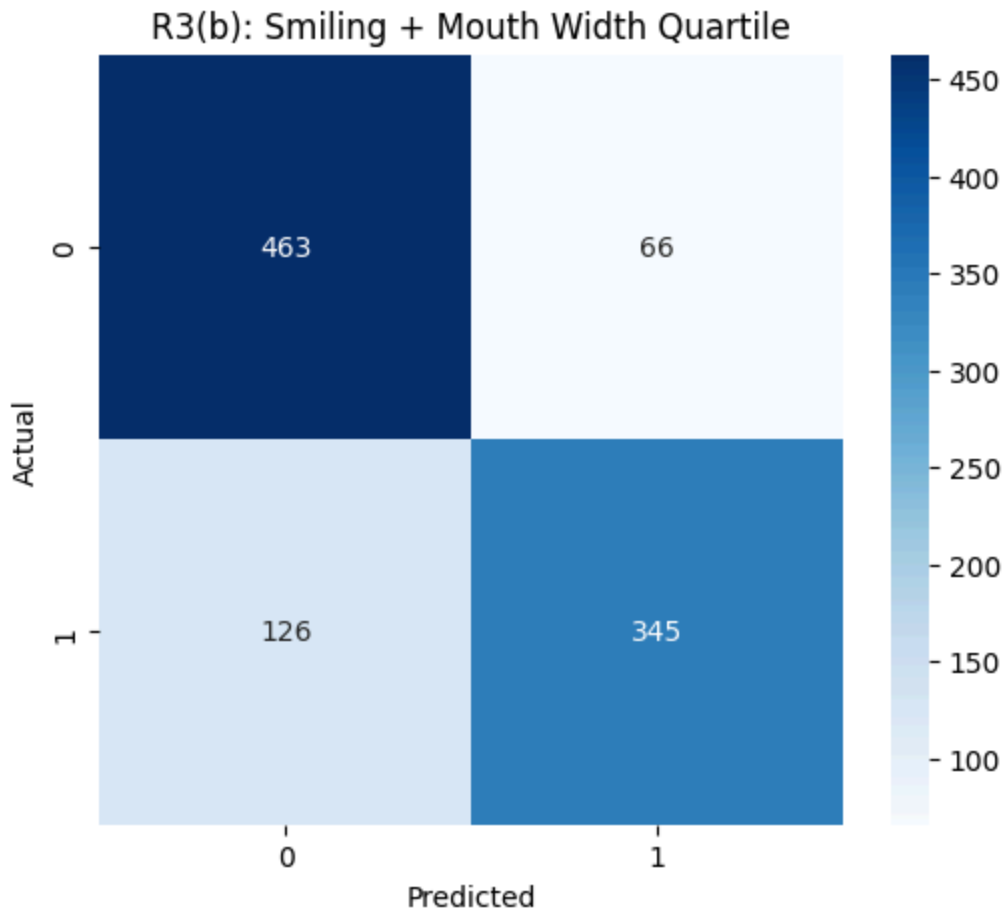
Trainable params: 15,861,505 (60.51 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [34]:
```
# Step 21: Training R3(b) model
r3b_history = r3b_model.fit(
    r3b_train_gen,
    validation_data=r3b_val_gen,
    epochs=5 )
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
509/509 ━━━━━━━━━━━━━━━━━━━━ 0s 206ms/step - accuracy: 0.6181 - loss: 0.6671
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_
adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**
kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing
`, `max_queue_size`. Do not pass these arguments to `fit()`, as they will be ignore
d.
  self._warn_if_super_not_called()
```

```
━━━
```



R3 (c) - Conclusion Comparison of basline accuracy from a with the model accuracy b. This is explanation of mouth width index.

In [43]:
```
# Step 23: Final explanation for evaluation of R3(c)
print(f"R3(a) baseline accuracy                 : {r3a_acc:.4f}")
print(f"R3(b) with mouth width quartile accuracy: {r3b_acc:.4f}")

if r3b_acc > r3a_acc:
    print("\nConclusion: mouth width quartile seems to be helpful metadata since it
elif r3b_acc < r3a_acc:
    print("\nConclusion: mouth width quartile does not seems to help on this proces
else:
    print("\nConclusion: mouth width quartile did not change test accuracy on this
```

```
R3(a) baseline accuracy                 : 0.7910
R3(b) with mouth width quartile accuracy: 0.8080

Conclusion: mouth width quartile seems to be helpful metadata since it is improved t
est accuracy.
```

In [44]:
```
!jupyter nbconvert --to html /content/ML_project_2.ipynb
```

```
[NbConvertApp] Converting notebook /content/ML_project_2.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 4 image(s).
[NbConvertApp] Writing 558229 bytes to /content/ML_project_2.html
```

Link of the colab - https://colab.research.google.com/drive/1SI6Jm_1HiVBydnF7cTSik1-uu6_G7Hhu?usp=sharing